



**UNIVERSITÉ HASSAN PREMIER SETTAT
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES -
BERRECHID**

Cycle Ingénieur
Ingénierie des Systèmes d'Information et Big Data (ISIBD)

**COMPTE RENDU DES TRAVAUX
PRATIQUES**

EN BASES DE DONNÉES AVANCÉES

Réalisé par :
Najat EL FAKIRI

Encadré par :
Mr.HRIMECH

Table des matières

1	Introduction	2
2	TP : Jointure & Sous-requêtes	3
2.1	Correction du TD de réversion (sous-requêtes)	3
2.1.1	Exercice 1 : Base de données ETUDIANTS	3
2.1.2	Exercice 2 : Base de données EMPLOYES	6
3	TP : Vues et Index en PL/SQL	8
3.1	Correction des exercices complémentaires (Jointures et Regroupements)	8
3.1.1	Exercice 3 : Gestion des Employés et Départements	8
3.1.2	Exercice 4 : Analyse Avancée et Rémunérations	10
4	Série d'exercices : Curseurs	11
4.1	Correction du TP : PL-SQL Curseur	11
4.1.1	Exercice 1 : Base de données Usine	11
4.1.2	Exercice 2 : Gestion de la société Gavasoftware	13
5	Série d'exercices : Procédures & Fonctions	15
5.1	Correction du TP 2 : Procédures stockées et fonctions en PL/SQL	15
5.1.1	Introduction et prérequis	15
5.1.2	Exercice 1 : Premiers pas avec les procédures et fonctions	15
5.1.3	Exercice 2 : Procédures applicatives sur la base ETUDIANTS	16
5.1.4	Exercice 3 : Gestion de la société (Procédures et Fonctions)	18
6	Série d'exercices : Les Triggers	20
6.1	Correction du TP : Les Triggers (Déclencheurs) en PL/SQL	20
6.1.1	Exercice 1 : Contrôle de cohérence des salaires	20
6.1.2	Exercice 2 : Initialisation automatique des commissions	20
6.1.3	Exercice 3 : Audit et Historisation des postes	21
6.1.4	Exercice 4 : Contrôle du budget global (Trigger de Table)	21
7	Conclusion et Remerciements	22
7.1	Conclusion	22
7.2	Remerciements	22

1. Introduction

Les systèmes de gestion de bases de données constituent aujourd'hui le pilier central de l'architecture et de la pérennité de toute application informatique moderne . L'exploitation et l'administration de ces réservoirs d'informations reposent traditionnellement sur le langage SQL (*Structured Query Language*), un standard universel qui offre la rigueur et l'efficacité nécessaires pour interroger, structurer et manipuler la donnée avec une haute précision .

Malgré l'émergence de nouveaux paradigmes, le modèle relationnel conserve une prédominance incontestée dans l'écosystème numérique en raison de sa clarté conceptuelle et de sa remarquable flexibilité adaptative . En agencant l'information sous forme de relations (tables) composées d'attributs (colonnes) et de n-uplets (lignes), il propose une représentation intuitive et mathématiquement saine des flux métiers . Par ailleurs, les processus de normalisation inhérents à ce modèle jouent un rôle critique : ils éliminent les redondances et structurent les dépendances fonctionnelles, garantissant ainsi l'intégrité globale et la cohérence absolue des données au sein des écosystèmes applicatifs .

Dans le cadre de ce compte rendu, nous mettrons en pratique ces théories fondamentales en nous immergeant dans l'environnement robuste d'Oracle XE Edition, interfacé avec SQL Developer, afin d'apporter des solutions concrètes et optimisées aux différents travaux pratiques détaillés dans notre sommaire .

2. TP : Jointure & Sous-requêtes

2.1 Correction du TD de réversion (sous-requêtes)

Ce document présente les solutions SQL optimales pour les exercices du TD, en mettant l'accent sur l'utilisation des sous-requêtes.

2.1.1 Exercice 1 : Base de données ETUDIANTS

Schéma relationnel :

- ETUDIANT (Numetu, Nometu, Dtnaiss, Cdsexe)
- SEXE (Cdsexe, Lbsexe)
- ENSEIGNANT (Numens, Nomens, Grade, Ancien)
- MATIERE (Numat, Nomat, Coeff, Numens)
- NOTES (Numetu, Numat, Note)

1. Question 1 : Numéro, nom et sexe des étudiants dont la date de naissance n'est pas connue

```
1 SELECT Numetu, Nometu, Cdsexe
2 FROM ETUDIANT
3 WHERE Dtnaiss IS NULL;
```

2. Question 2 : Nom et date de naissance des étudiants dont la 2^e lettre du nom est 'a'

```
1 SELECT Nometu, Dtnaiss
2 FROM ETUDIANT
3 WHERE Nometu LIKE '_a%';
```

3. Question 3 : Nom des matières qui ont un coefficient compris entre 1 et 2

```
1 SELECT Nomat
2 FROM MATIERE
3 WHERE Coeff BETWEEN 1 AND 2;
```

4. Question 4 : Afficher les notes de l'étudiant numéro 1 qui sont égales aux notes de l'étudiant numéro 2

```
1 SELECT Note, Numat
2 FROM NOTES
3 WHERE Numetu = 1
4     AND Note IN (
5         SELECT Note
6         FROM NOTES
7         WHERE Numetu = 2
8     );
```

5. Question 5 : Afficher les notes de l'étudiant numéro 1 qui sont supérieures aux notes de l'étudiant numéro 2

```

1  -- Cas 1 : Supérieure a TOUTES les notes de l'etudiant 2
2  SELECT Note, Numat
3  FROM NOTES
4  WHERE Numetu = 1
5         AND Note > ALL (
6             SELECT Note
7             FROM NOTES
8             WHERE Numetu = 2
9         );
10
11 -- Cas 2 : Supérieure a AU MOINS UNE des notes de l'etudiant 2
12 SELECT Note, Numat
13 FROM NOTES
14 WHERE Numetu = 1
15        AND Note > ANY (
16            SELECT Note
17            FROM NOTES
18            WHERE Numetu = 2
19        );

```

6. Question 6 : Afficher les notes de l'étudiant numéro 1 qui sont inférieures à toutes les notes de l'étudiant numéro 9

```

1  SELECT Note, Numat
2  FROM NOTES
3  WHERE Numetu = 1
4         AND Note < ALL (
5             SELECT Note
6             FROM NOTES
7             WHERE Numetu = 9
8         );

```

7. Question 7 : Afficher toutes les informations sur les étudiants qui n'ont aucune note

```

1  SELECT *
2  FROM ETUDIANT
3  WHERE Numetu NOT IN (
4      SELECT Numetu
5      FROM NOTES
6  );

```

8. Question 8 : Afficher quel était l'âge moyen des garçons et des filles au premier janvier 2000

```

1  SELECT Cdsexe, AVG(2000 - EXTRACT(YEAR FROM Dtnaiss)) AS AgeMoyen
2  FROM ETUDIANT
3  WHERE Dtnaiss IS NOT NULL
4  GROUP BY Cdsexe;

```

9. Question 9 : Afficher le nom et le grade des enseignants d'histoire

```

1  SELECT Nomens, Grade
2  FROM ENSEIGNANT
3  WHERE Numens IN (
4      SELECT Numens

```

```
5      FROM MATIERE
6      WHERE UPPER(Nomat) LIKE '%HISTOIRE%'
7  );
```

10. Question 10 : Afficher les noms et numéro des étudiants qui n'ont pas de notes en Sociologie

```
1  SELECT Numetu, Nometu
2  FROM ETUDIANT
3  WHERE Numetu NOT IN (
4      SELECT Numetu
5      FROM NOTES
6      WHERE Numat IN (
7          SELECT Numat
8          FROM MATIERE
9          WHERE Nomat = 'Sociologie'
10     )
11 );
```

2.1.2 Exercice 2 : Base de données EMPLOYES

Schéma relationnel :

- EMP (Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept)
- DEPT (NumDept, NomDept, Lieu)
- PROJET (CodeP, NomP)
- PARTICIPATION (Matr, CodeP, Fonction)

11. Question 11 : Lieu des départements dans lesquels des employés touchent une commission, en utilisant cette fois une sous-interrogation et non plus une jointure

```

1 SELECT Lieu
2 FROM DEPT
3 WHERE NumDept IN (
4     SELECT NumDept
5     FROM EMP
6     WHERE Commission IS NOT NULL AND Commission > 0
7 );

```

12. Question 12 : Noms et lieux des départements dans lesquels il y a au moins un ingénieur. Pouvez-vous y arriver au moyen d'une jointure ?

Réponse : Oui, la jointure classique est tout à fait possible et souvent plus intuitive pour ce cas précis.

```

1 -- Methode 1 : Avec sous-interrogation (recommandee pour ce TD)
2 SELECT NomDept, Lieu
3 FROM DEPT
4 WHERE NumDept IN (
5     SELECT NumDept
6     FROM EMP
7     WHERE Poste = 'Ingenieur'
8 );
9
10 -- Methode 2 : Avec une jointure explicite
11 SELECT DISTINCT D.NomDept, D.Lieu
12 FROM DEPT D
13 JOIN EMP E ON D.NumDept = E.NumDept
14 WHERE E.Poste = 'Ingenieur';

```

13. Question 13 : Noms et lieux des départements dans lesquels il n'y a pas d'ingénieur

```

1 SELECT NomDept, Lieu
2 FROM DEPT
3 WHERE NumDept NOT IN (
4     SELECT NumDept
5     FROM EMP
6     WHERE Poste = 'Ingenieur'
7     AND NumDept IS NOT NULL
8 );

```

14. Question 14 : À l'aide d'une sous-interrogation synchronisée, dressez la liste des employés qui gagnent moins de 50 % du salaire de leur supérieur direct

```

1 SELECT *
2 FROM EMP E
3 WHERE Salaire < 0.5 * (

```

```

4      SELECT Salaire
5      FROM EMP M
6      WHERE M.Matr = E.Sup
7  );

```

15. Question 15 : Noms des employés qui gagnent plus que tous les commerciaux

```

1  SELECT NomE
2  FROM EMP
3  WHERE Salaire > ALL (
4      SELECT Salaire
5      FROM EMP
6      WHERE Poste = 'Commercial'
7  );

```

16. Question 16 : Noms des employés qui gagnent plus que tous les commerciaux de leur département.

— Solution incluant les employés dans un département sans aucun commercial :

```

1  SELECT NomE
2  FROM EMP E
3  WHERE Salaire > ALL (
4      SELECT Salaire
5      FROM EMP C
6      WHERE C.Poste = 'Commercial'
7      AND C.NumDept = E.NumDept
8  );

```

— Solution excluant les employés dans un département sans commercial :

```

1  SELECT NomE
2  FROM EMP E
3  WHERE Salaire > ALL (
4      SELECT Salaire
5      FROM EMP C
6      WHERE C.Poste = 'Commercial'
7      AND C.NumDept = E.NumDept
8  )
9  AND EXISTS (
10     SELECT 1
11     FROM EMP C2
12     WHERE C2.Poste = 'Commercial'
13     AND C2.NumDept = E.NumDept
14 );

```

17. Question 17 : En utilisant une sous-interrogation à plusieurs colonnes, dressez la liste des employés ayant le même poste et le même supérieur que BERGER

```

1  SELECT NomE
2  FROM EMP
3  WHERE (Poste, Sup) IN (
4      SELECT Poste, Sup
5      FROM EMP
6      WHERE NomE = 'BERGER'
7  )
8  AND NomE <> 'BERGER';

```


3. TP : Vues et Index en PL/SQL

3.1 Correction des exercices complémentaires (Jointures et Regroupements)

Cette section regroupe les solutions SQL pour les exercices portant sur les jointures complexes, les critères d'agrégation et les sous-requêtes corrélées.

3.1.1 Exercice 3 : Gestion des Employés et Départements

1. Question 1 : Lieu des départements dans lesquels des employés touchent une commission

```
1 SELECT DISTINCT Lieu
2 FROM DEPT D
3 JOIN EMP E ON D.NumDept = E.NumDept
4 WHERE E.Commission IS NOT NULL
5        AND E.Commission > 0;
```

2. Question 2 : Noms et lieux des départements dans lesquels il y a au moins un ingénieur

```
1 SELECT DISTINCT NomDept, Lieu
2 FROM DEPT D
3 JOIN EMP E ON D.NumDept = E.NumDept
4 WHERE E.Poste = 'Ingenieur';
```

3. Question 3 : Noms et lieux des départements dans lesquels il n'y a pas d'ingénieur

```
1 SELECT NomDept, Lieu
2 FROM DEPT
3 WHERE NumDept NOT IN (
4     SELECT DISTINCT NumDept
5     FROM EMP
6     WHERE Poste = 'Ingenieur'
7         AND NumDept IS NOT NULL
8 );
```

4. Question 4 : Liste des employés qui gagnent moins de 50 % du salaire de leur supérieur direct

```
1 SELECT E.NomE, E.Salaire
2 FROM EMP E
3 JOIN EMP M ON E.Sup = M.Matr
4 WHERE E.Salaire < 0.5 * M.Salaire;
```

5. Question 5 : Noms des employés qui gagnent plus que tous les commerciaux

```
1 SELECT NomE
2 FROM EMP
3 WHERE Salaire > ALL (
4     SELECT Salaire
5     FROM EMP
6     WHERE Poste = 'Commercial'
7 );
```

3.1.2 Exercice 4 : Analyse Avancée et Rémunérations

6. Question 6 : Noms des employés qui gagnent plus que tous les commerciaux de leur département

```
1 -- Solution incluant les departements sans commerciaux
2 SELECT NomE
3 FROM EMP E
4 WHERE Salaire > ALL (
5     SELECT Salaire
6     FROM EMP C
7     WHERE C.Poste = 'Commercial'
8     AND C.NumDept = E.NumDept
9 );
```

7. Question 7 : Liste des employés ayant le même poste et le même supérieur que BERGER

```
1 SELECT NomE
2 FROM EMP
3 WHERE (Poste, Sup) IN (
4     SELECT Poste, Sup
5     FROM EMP
6     WHERE NomE = 'BERGER'
7 )
8 AND NomE <> 'BERGER';
```

8. Question 8 : Nom des départements et salaire moyen par département

```
1 SELECT D.NomDept, AVG(E.Salaire) AS SalaireMoyen
2 FROM DEPT D
3 LEFT JOIN EMP E ON D.NumDept = E.NumDept
4 GROUP BY D.NomDept, D.NumDept;
```

9. Question 9 : Numéro et nom des départements qui ont plus de 3 ingénieurs

```
1 SELECT D.NumDept, D.NomDept
2 FROM DEPT D
3 JOIN EMP E ON D.NumDept = E.NumDept
4 WHERE E.Poste = 'Ingenieur'
5 GROUP BY D.NumDept, D.NomDept
6 HAVING COUNT(*) > 3;
```

10. Question 10 : Nom des employés travaillant sur un projet à Paris

```
1 SELECT DISTINCT E.NomE
2 FROM EMP E
3 JOIN PARTICIPATION PA ON E.Matr = PA.Matr
4 JOIN PROJET PR ON PA.CodeP = PR.CodeP
5 JOIN DEPT D ON E.NumDept = D.NumDept
6 WHERE D.Lieu = 'Paris';
```

4. Série d'exercices : Curseurs

4.1 Correction du TP : PL-SQL Curseur

Cette section regroupe les scripts PL/SQL répondant aux exercices de manipulation des curseurs explicites et des boucles de traitement de données (FOR et FETCH).

4.1.1 Exercice 1 : Base de données Usine

Schéma relationnel :

- Commande (Numcmd, Datecmd, #Numfour, Mncmd)
- Fournisseurs (Numfour, Adrfour)

1. Question 1 : Donnez la liste des commandes

```
1 DECLARE
2     -- Definition du curseur pour lister toutes les commandes
3     CURSOR c_commandes IS
4         SELECT Numcmd, Datecmd, Numfour, Mncmd FROM Commande;
5     v_cmd c_commandes%ROWTYPE;
6 BEGIN
7     OPEN c_commandes;
8     LOOP
9         FETCH c_commandes INTO v_cmd;
10        EXIT WHEN c_commandes%NOTFOUND;
11        DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_cmd.Numcmd ||
12                               ' - Date: ' || v_cmd.Datecmd ||
13                               ' - Fournisseur: ' || v_cmd.Numfour
14                               ' - Montant: ' || v_cmd.Mncmd);
15    END LOOP;
16    CLOSE c_commandes;
17 END;
18 /
```

2. Question 2 : Donnez la liste des commandes ayant un montant supérieur à la moyenne

```
1 DECLARE
2     CURSOR c_cmd_sup_moy IS
3         SELECT Numcmd, Mncmd
4         FROM Commande
5         WHERE Mncmd > (SELECT AVG(Mncmd) FROM Commande);
6     v_cmd c_cmd_sup_moy%ROWTYPE;
7 BEGIN
8     OPEN c_cmd_sup_moy;
9     LOOP
10        FETCH c_cmd_sup_moy INTO v_cmd;
11        EXIT WHEN c_cmd_sup_moy%NOTFOUND;
```

```
12         DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_cmd.Numcmd ||  
13                               ' - Montant: ' || v_cmd.Mncmd);  
14     END LOOP;  
15     CLOSE c_cmd_sup_moy;  
16 END;  
17 /
```

3. Question 3 : Donnez la liste des commandes ayant un fournisseur parisien

```
1 DECLARE  
2     CURSOR c_cmd_paris IS  
3         SELECT C.Numcmd, C.Mncmd, F.Adrfour  
4         FROM Commande C  
5         JOIN Fournisseurs F ON C.Numfour = F.Numfour  
6         WHERE UPPER(F.Adrfour) LIKE '%PARIS%';  
7     v_line c_cmd_paris%ROWTYPE;  
8 BEGIN  
9     FOR v_line IN c_cmd_paris LOOP  
10        DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_line.Numcmd ||  
11                              ' - Montant: ' || v_line.Mncmd ||  
12                              ' - Ville: ' || v_line.Adrfour);  
13    END LOOP;  
14 END;  
15 /
```

4.1.2 Exercice 2 : Gestion de la société Gavasoft

L'énoncé impose pour cet exercice l'utilisation de la structure de boucle FOR (gestion semi-automatique des curseurs).

1. Question 1 : Donnez la liste des employés ayant une commission (non NULL) classée par commission décroissante

```

1 DECLARE
2     CURSOR c_emp_comm IS
3         SELECT NomE, Comm
4         FROM Emp
5         WHERE Comm IS NOT NULL AND Comm > 0
6         ORDER BY Comm DESC;
7 BEGIN
8     FOR r_emp IN c_emp_comm LOOP
9         DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE ||
10                                ' - Commission: ' || r_emp.Comm);
11     END LOOP;
12 END;
13 /

```

2. Question 2 : Donnez les noms des personnes embauchées depuis le 01-09-2006

```

1 DECLARE
2     CURSOR c_emp_recents IS
3         SELECT NomE, Embauche
4         FROM Emp
5         WHERE Embauche >= TO_DATE('01/09/2006', 'DD/MM/YYYY');
6 BEGIN
7     FOR r_emp IN c_emp_recents LOOP
8         DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE ||
9                                ' - Date embauche: ' || TO_CHAR(r_emp
10                                .Embauche, 'DD/MM/YYYY'));
11     END LOOP;
12 END;
13 /

```

3. Question 3 : Donnez la liste des employés travaillant à Créteil

```

1 DECLARE
2     CURSOR c_emp_creteil IS
3         SELECT E.NomE, D.NomD, D.Lieu
4         FROM Emp E
5         JOIN Dept D ON E.NumD = D.NumD
6         WHERE UPPER(D.Lieu) = 'CR TEIL' OR UPPER(D.Lieu) = '
7             CRETEIL';
8 BEGIN
9     FOR r_emp IN c_emp_creteil LOOP
10         DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE ||
11                                ' - D departement: ' || r_emp.NomD);
12     END LOOP;
13 END;
14 /

```

4. Question 4 : Donnez la liste des subordonnés de "Guimezanes"

```

1 DECLARE
2     CURSOR c_subordonnes IS
3         SELECT NomE, Fonction

```

```

4      FROM Emp
5      WHERE NumS = (SELECT Matr FROM Emp WHERE NomE = '
      Guimezanes');
6 BEGIN
7     FOR r_emp IN c_subordonnes LOOP
8         DBMS_OUTPUT.PUT_LINE('Subordonn : ' || r_emp.NomE ||
9                                ' - Fonction: ' || r_emp.Fonction);
10    END LOOP;
11 END;
12 /

```

5. Question 5 : Donnez la moyenne des salaires

```

1 DECLARE
2     v_moyenne NUMBER;
3 BEGIN
4     SELECT AVG(Salaire) INTO v_moyenne FROM Emp;
5     DBMS_OUTPUT.PUT_LINE('La moyenne globale des salaires est de :
6                             ' || ROUND(v_moyenne, 2));
7 END;
8 /

```

6. Question 6 : Donnez le nombre de commissions non NULL

```

1 DECLARE
2     v_nb_comm NUMBER;
3 BEGIN
4     SELECT COUNT(Comm) INTO v_nb_comm FROM Emp WHERE Comm IS NOT
5     NULL;
6     DBMS_OUTPUT.PUT_LINE('Nombre d''employ s touchant une
7                             commission : ' || v_nb_comm);
8 END;
9 /

```

7. Question 7 : Donnez la liste des employés gagnant plus que la moyenne des salaires de l'entreprise

```

1 DECLARE
2     CURSOR c_emp_riches IS
3         SELECT NomE, Salaire
4         FROM Emp
5         WHERE Salaire > (SELECT AVG(Salaire) FROM Emp);
6 BEGIN
7     FOR r_emp IN c_emp_riches LOOP
8         DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE ||
9                                ' - Salaire: ' || r_emp.Salaire);
10    END LOOP;
11 END;
12 /

```

5. Série d'exercices : Procédures & Fonctions

5.1 Correction du TP 2 : Procédures stockées et fonctions en PL/SQL

Cette section présente les solutions sous forme de blocs de code et de sous-programmes (procédures et fonctions) développés en PL/SQL.

5.1.1 Introduction et prérequis

Avant de créer des procédures ou fonctions stockées, il est nécessaire d'activer l'affichage des sorties serveurs et de s'assurer de la bonne structure des tables.

```
1 -- Activation de l'affichage
2 SET SERVEROUTPUT ON;
```

5.1.2 Exercice 1 : Premiers pas avec les procédures et fonctions

1. Question 1 : Procédure simple d'affichage ("Bonjour")

```
1 CREATE OR REPLACE PROCEDURE AfficherBonjour IS
2 BEGIN
3     DBMS_OUTPUT.PUT_LINE('Bonjour tout le monde !');
4 END;
5 /
6 -- Appel de la procedure :
7 EXEC AfficherBonjour;
```

2. Question 2 : Fonction de calcul du carré d'un nombre

```
1 CREATE OR REPLACE FUNCTION CalculerCarre(p_nombre NUMBER)
2 RETURN NUMBER IS
3 BEGIN
4     RETURN p_nombre * p_nombre;
5 END;
6 /
7 -- Exemple d'appel :
8 SELECT CalculerCarre(5) FROM DUAL;
```


5.1.3 Exercice 2 : Procédures applicatives sur la base ETUDIANTS

3. Question 3 : Procédure affichant les informations d'un étudiant à partir de son numéro

```

1 CREATE OR REPLACE PROCEDURE InfoEtudiant(p_numetu ETUDIANT.Nometu%
  TYPE) IS
2     v_nom      ETUDIANT.Nometu%TYPE;
3     v_date     ETUDIANT.Dtnaiss%TYPE;
4     v_sexe     ETUDIANT.Cdsexe%TYPE;
5 BEGIN
6     SELECT Nometu, Dtnaiss, Cdsexe
7     INTO v_nom, v_date, v_sexe
8     FROM ETUDIANT
9     WHERE Nometu = p_numetu;
10
11     DBMS_OUTPUT.PUT_LINE('Nom : ' || v_nom ||
12                          ' | Date de Naissance : ' || TO_CHAR(
13                          v_date, 'DD/MM/YYYY') ||
14                          ' | Code Sexe : ' || v_sexe);
15 EXCEPTION
16     WHEN NO_DATA_FOUND THEN
17         DBMS_OUTPUT.PUT_LINE('Erreur : Aucun tudiant trouv
18         avec le num ro ' || p_numetu);
19 END;
20 /

```

4. Question 4 : Fonction renvoyant le nombre de notes d'un étudiant donné

```

1 CREATE OR REPLACE FUNCTION NbNotesEtudiant(p_numetu NOTES.Nometu%
  TYPE)
2 RETURN NUMBER IS
3     v_count NUMBER := 0;
4 BEGIN
5     SELECT COUNT(*)
6     INTO v_count
7     FROM NOTES
8     WHERE Nometu = p_numetu;
9
10    RETURN v_count;
11 END;
12 /

```

5. Question 5 : Procédure calculant et affichant la moyenne d'un étudiant

```

1 CREATE OR REPLACE PROCEDURE MoyenneEtudiant(p_numetu NOTES.Nometu%
  TYPE) IS
2     v_moyenne NUMBER;
3 BEGIN
4     SELECT AVG(Note)
5     INTO v_moyenne
6     FROM NOTES
7     WHERE Nometu = p_numetu;
8
9     IF v_moyenne IS NOT NULL THEN
10         DBMS_OUTPUT.PUT_LINE('Moyenne de l'' tudiant ' ||
11                                p_numetu || ' : ' || ROUND(v_moyenne, 2));
12     ELSE
13         DBMS_OUTPUT.PUT_LINE('L'' tudiant ' || p_numetu || ' n''a
14         pas encore de notes.');
```

```
13     END IF;  
14 END;  
15 /
```

5.1.4 Exercice 3 : Gestion de la société (Procédures et Fonctions)

6. Question 6 : Fonction retournant le salaire annuel d'un employé (Salaire $\times$ 12 + Commission)

```

1 CREATE OR REPLACE FUNCTION SalaireAnnuel(p_matr EMP.Matr%TYPE)
2 RETURN NUMBER IS
3     v_salaire EMP.Salaire%TYPE;
4     v_commission EMP.Commission%TYPE;
5 BEGIN
6     SELECT Salaire, NVL(Commission, 0)
7     INTO v_salaire, v_commission
8     FROM EMP
9     WHERE Matr = p_matr;
10
11     RETURN (v_salaire * 12) + v_commission;
12 EXCEPTION
13     WHEN NO_DATA_FOUND THEN
14         RETURN 0;
15 END;
16 /

```

7. Question 7 : Procédure d'augmentation de salaire pour les employés d'un département donné

```

1 CREATE OR REPLACE PROCEDURE AugmenterSalaireDept(
2     p_numdept EMP.NumDept%TYPE,
3     p_pourcentage NUMBER
4 ) IS
5 BEGIN
6     UPDATE EMP
7     SET Salaire = Salaire * (1 + p_pourcentage / 100)
8     WHERE NumDept = p_numdept;
9
10    DBMS_OUTPUT.PUT_LINE('Mise      jour effectu e. ' || SQL%
11    ROWCOUNT || ' employ (s) augment (s).');
12    COMMIT;
13 END;
14 /

```

8. Question 8 : Procédure affichant la liste des subordonnés d'un responsable à l'aide d'un curseur

```

1 CREATE OR REPLACE PROCEDURE ListeSubordonnes(p_sup EMP.Matr%TYPE)
2 IS
3     CURSOR c_sub IS
4         SELECT NomE, Poste
5         FROM EMP
6         WHERE Sup = p_sup;
7     v_vide BOOLEAN := TRUE;
8 BEGIN
9     DBMS_OUTPUT.PUT_LINE('Liste des subordonn s du responsable '
10 || p_sup || ' :');
11     FOR r_emp IN c_sub LOOP
12         v_vide := FALSE;
13         DBMS_OUTPUT.PUT_LINE('- ' || r_emp.NomE || ' (Poste : ' ||
14         r_emp.Poste || ')');
15     END LOOP;
16

```

```
14      IF v_vide THEN
15          DBMS_OUTPUT.PUT_LINE('Cet employ n''a aucun subordonn
16              sa charge. ');
17      END IF;
18  END;
19  /
```

6. Série d'exercices : Les Triggers

6.1 Correction du TP : Les Triggers (Déclencheurs) en PL/SQL

Cette section présente les scripts de création de déclencheurs (triggers) permettant d'automatiser les contrôles d'intégrité, la gestion des historiques et l'application des règles de gestion métiers de l'entreprise.

6.1.1 Exercice 1 : Contrôle de cohérence des salaires

1. Question : Écrire un trigger qui empêche de diminuer le salaire d'un employé lors d'une modification.

```
1 CREATE OR REPLACE TRIGGER trg_verif_salaire
2 BEFORE UPDATE OF Salaire ON EMP
3 FOR EACH ROW
4 BEGIN
5     -- Verification de la non-regression du salaire
6     IF :NEW.Salaire < :OLD.Salaire THEN
7         RAISE_APPLICATION_ERROR(-20001, 'Erreur : Le salaire ne
8         peut pas tre diminue . Salaire actuel : ' || :OLD.Salaire);
9     END IF;
10 END;
```

6.1.2 Exercice 2 : Initialisation automatique des commissions

2. Question : Écrire un trigger qui attribue automatiquement une commission de 0 à la création d'un employé si aucune valeur n'est spécifiée.

```
1 CREATE OR REPLACE TRIGGER trg_default_commission
2 BEFORE INSERT ON EMP
3 FOR EACH ROW
4 BEGIN
5     IF :NEW.Commission IS NULL THEN
6         :NEW.Commission := 0;
7     END IF;
8 END;
```

6.1.3 Exercice 3 : Audit et Historisation des postes

Prérequis : Création de la table d'archivage ou d'audit des mouvements de postes.

```

1 CREATE TABLE Historique_Poste (
2     Matr NUMBER,
3     Ancien_Poste VARCHAR2(50),
4     Nouveau_Poste VARCHAR2(50),
5     Date_Changement DATE,
6     Utilisateur VARCHAR2(50)
7 );

```

3. Question : Écrire un trigger qui enregistre l'historique des changements de poste d'un employé dans la table Historique_Poste.

```

1 CREATE OR REPLACE TRIGGER trg_historique_poste
2 AFTER UPDATE OF Poste ON EMP
3 FOR EACH ROW
4 BEGIN
5     INSERT INTO Historique_Poste (
6         Matr,
7         Ancien_Poste,
8         Nouveau_Poste,
9         Date_Changement,
10        Utilisateur
11    ) VALUES (
12        :OLD.Matr,
13        :OLD.Poste,
14        :NEW.Poste,
15        SYSDATE,
16        USER
17    );
18 END;
19 /

```

6.1.4 Exercice 4 : Contrôle du budget global (Trigger de Table)

4. Question : Écrire un déclencheur au niveau de la table (Statement-level) qui interdit toute augmentation si la masse salariale totale de l'entreprise dépasse un plafond critique (ex : 500 000).

```

1 CREATE OR REPLACE TRIGGER trg_masse_salariale_plafond
2 AFTER INSERT OR UPDATE OF Salaire ON EMP
3 DECLARE
4     v_masse_totale NUMBER;
5     v_plafond      CONSTANT NUMBER := 500000;
6 BEGIN
7     -- Calcul de la nouvelle masse salariale globale
8     SELECT SUM(Salaire) INTO v_masse_totale FROM EMP;
9
10    IF v_masse_totale > v_plafond THEN
11        RAISE_APPLICATION_ERROR(-20002, 'Action refusée : Le
12        budget total maximal alloué aux salaires (' || v_plafond || ')
13        serait dépassé.');
14    END IF;
15 END;
16 /

```

““

7. Conclusion et Remerciements

7.1 Conclusion

La réalisation de ces différents travaux pratiques a constitué une opportunité précieuse pour appréhender et maîtriser les concepts fondamentaux du langage PL/SQL sous Oracle. L'exploration combinée des curseurs, des mécanismes de gestion des exceptions, ainsi que des sous-programmes stockés (fonctions et procédures) nous a permis d'acquérir une expertise solide en matière de développement et d'optimisation de bases de données relationnelles.

D'une part, l'implémentation des curseurs a facilité le traitement structuré d'ensembles de données complexes, tandis que la configuration rigoureuse des exceptions a garanti la robustesse, la tolérance aux pannes et la fiabilité de nos scripts . D'autre part, la modularité offerte par les fonctions et les procédures a mis en évidence les gains évidents en termes de réutilisabilité, de clarté et de maintenance du code applicatif . Ce parcours d'apprentissage, mené à bien au sein d'un environnement moderne combinant Oracle Database 21c et l'interface SQL Developer, nous dote désormais des compétences et de l'assurance nécessaires pour concevoir des architectures de données performantes et répondre efficacement aux exigences du développement back-end .

7.2 Remerciements

Nous tenons à exprimer notre profonde et sincère gratitude à notre professeur, M. Hamid Hrimech, pour son accompagnement bienveillant, son encadrement de grande qualité et son investissement constant tout au long de ce module dédié au PL/SQL .

Nous le remercions chaleureusement pour la clarté de ses enseignements, sa disponibilité intellectuelle et la passion pédagogique qu'il a transmise à notre groupe de travail. Les compétences techniques et méthodologiques acquises sous sa direction représentent un atout indéniable, qui saura sans nul doute valoriser et consolider notre futur parcours professionnel . Que M. Hrimech trouve ici le témoignage de notre haute considération et de nos remerciements les plus sincères pour son engagement indéfectible envers notre réussite universitaire .